

Rust-এর ডাটা টাইপগুলোকে (Data Types) প্রধানত দুটি ভাগে ভাগ করা যায়: [1, 2]

1. **Scalar Types** (যা একটি মাত্র সিস্টেম ভ্যালু রিপ্রেজেন্ট করে)
2. **Compound Types** (যা একাধিক ভ্যালুকে একসাথে গ্রুপ করে) [3, 4]

নিচে সহজ ভাষায় উদাহরণসহ প্রতিটির বিবরণ দেওয়া হলো:

১. Scalar Types (স্কেলার টাইপ)

এর মধ্যে ৪টি মূল ক্যাটাগরি আছে:

- **Integer (পূর্ণসংখ্যা):** ভগ্নাংশ ছাড়া যেকোনো সংখ্যা। এগুলো আবার দুই ধরনের হয়:
 - **Signed (i8, i16, i32, i64, i128):** পজিটিভ এবং নেগেটিভ দুই ধরনের সংখ্যাই সাপোর্ট করে।
 - **Unsigned (u8, u16, u32, u64, u128):** শুধু পজিটিভ সংখ্যা সাপোর্ট করে।
 - **নোট:** Rust-এ ডিফল্ট ইন্টিজার টাইপ হলো i32। [5, 6, 7, 8, 9]
- **Floating-Point (দশমিক সংখ্যা):** দশমিক যুক্ত সংখ্যা।
 - **f32:** সিস্টেম-প্রিসিশন দশমিক।
 - **f64:** ডাবল-প্রিসিশন দশমিক (এটি Rust-এর ডিফল্ট এবং আধুনিক কম্পিউটারে বেশি ফাস্ট)। [10]
- **Boolean (বুলিয়ান):** শুধু দুটি ভ্যালু হতে পারে।
 - **true** (সত্য) অথবা **false** (মিথ্যা)। টাইপ লেখার সময় bool ব্যবহার করা হয়। [11]
- **Character (ক্যারেক্টার):** যেকোনো একটি একক অক্ষর, সংখ্যা বা ইমোজি।
 - টাইপ লেখার সময় char ব্যবহার করা হয়।
 - ভ্যালুটিকে অবশ্যই single quote (' ') এর মধ্যে রাখতে হয় (যেমন: let letter = 'A'); [12, 13]

২. Compound Types (কম্পাউন্ড টাইপ)

একাধিক ভ্যালুকে একটি টাইপের মধ্যে রাখার জন্য এগুলো ব্যবহৃত হয়:

- **Tuple (টুপল):**
 - বিভিন্ন ধরনের (different types) ডাটা টাইপকে একসাথে গ্রুপ করা যায়।
 - এর সাইজ ফিক্সড, একবার ডিক্লেয়ার করলে আর বাড়ানো বা কমানো যায় না।
 - ফার্স্ট ব্র্যাকেট () দিয়ে লিখতে হয়।
 - **উদাহরণ:** let person: (String, i32) = ("Rahul".to_string(), 25); [14]
 - **Array (অ্যারে):**
 - একই ধরনের (same type) একাধিক ডাটা রাখার জন্য ব্যবহৃত হয়।
 - এর সাইজও ফিক্সড।
 - থার্ড ব্র্যাকেট [] দিয়ে লিখতে হয়।
 - **উদাহরণ:** let numbers: [i32; 3] = [10, 20, 30];
-

একটি ছোট কোড উদাহরণ:

```
fn main() {  
    // Scalar types  
    let age: u8 = 25;           // Unsigned integer  
    let pi: f64 = 3.1416;       // Floating-point  
    let is_rust_fun: bool = true; // Boolean  
    let grade: char = 'A';      // Character  
  
    // Compound types  
    let user: (&str, i32) = ("Abhi", 30); // Tuple  
    let scores: [i32; 4] = []; // Array  
}
```

Rust একটি **Statically Typed** প্রোগ্রামিং ল্যাঙ্গুয়েজ। এর মানে হলো, কোড রান (Run) করার আগেই, অর্থাৎ কমপাইল করার সময়ই Compiler-কে জানতে হবে প্রতিটি ভ্যারিয়েবলের ডাটা টাইপ কী।

তবে কোড লেখার সময় আপনাকে প্রতিবার টাইপ উল্লেখ করতে হয় না। Rust-এর চমৎকার একটি বৈশিষ্ট্য হলো **Type Inference**।

নিচে সহজ ভাষায় বুঝিয়ে দেওয়া হলো কীভাবে এটি কাজ করে:

১. Type Inference (টাইপ অনুমান করা)

আপনি যখন কোনো ভ্যারিয়েবলের টাইপ নিজে থেকে লিখে দেন না, তখন Rust Compiler তার ভ্যালু দেখে অটোমেটিক বুঝে নেয় সেটির টাইপ কী হওয়া উচিত।

- **উদাহরণ ১ (টাইপ ছাড়া):**

- `let x = 10;`

এখানে আপনি বলেননি `x` একটি ইন্টিজার। কিন্তু Rust কমপাইল করার সময় ভ্যালু 10 দেখে নিজে থেকেই `x`-কে `i32` (ডিফল্ট ইন্টিজার) টাইপ হিসেবে ধরে নেবে।

- **উদাহরণ ২ (টাইপ ছাড়া):**

- `let y = 5.5;`

এখানে ভ্যালু 5.5 দেখে Rust বুঝে নেবে এটি একটি `f64` (ডিফল্ট ফ্লোট) টাইপ।

২. Explicit Type Annotation (নিজে টাইপ বলে দেওয়া)

আপনি চাইলে নিজেই ভ্যারিয়েবলের টাইপ ফিক্সড করে দিতে পারেন। একে বলে Explicit Annotation। এটি কোলন (`:`) ব্যবহার করে করতে হয়।

- **উদাহরণ:**

- `let x: u64 = 10; // এখানে x-কে নির্দিষ্টভাবে 64-bit unsigned integer বলা হয়েছে`
- `let y: f32 = 5.5; // এখানে y-কে 32-bit float বলা হয়েছে`

Statically Typed হওয়ার সুবিধা কী?

- **কোড রান হওয়ার আগেই ভুল ধরা পড়ে:** আপনি যদি একটি ইন্টিজার টাইপ ভ্যারিয়েবলের সাথে টেক্সট বা ক্যারেক্টার যোগ করতে যান, Rust কোড রান হতেই দেবে না। কমপাইল করার সময়ই এরর (Error) দেখাবে।

- **সুপার ফাস্ট পারফরম্যান্স:** রানটাইমে (ল্যাপটপ বা সার্ভারে প্রোগ্রামটি চলার সময়) কম্পিউটারকে বারবার চেক করতে হয় না কোন ভ্যারিয়েবল কী টাইপের। কমপাইল করার সময়ই সব ঠিক হয়ে যাওয়ায় অ্যাপ অনেক ফাস্ট চলে।
- **কোনো রানটাইম ক্র্যাশ নেই:** অন্যান্য ডাইনামিক ল্যাঙ্গুয়েজের (যেমন JavaScript বা Python) মতো ডাটা টাইপের ভুলের কারণে মাঝপথে অ্যাপ ক্র্যাশ করার ভয় থাকে না।

একটি বাস্তব উদাহরণ যেখানে Compiler টাইপ বুঝে নেয়:

```
fn main() {
```

```
    let mut numbers = Vec::new(); // এখানে Rust এখনও জানে না এটা কিসের ভেক্টর (লিস্ট)
```

```
    numbers.push(5); // যখনই আপনি ৫ পুশ করলেন, Rust বুঝে গেল এটি ইন্টিজারের ভেক্টর!
```

```
    // numbers.push("hello"); // এই লাইনের কমেন্ট সরালে Rust এরর দেবে, কারণ সে অলরেডি এটাকে  
    // ইন্টিজার টাইপ ধরে নিয়েছে।
```

```
}
```

Rust-এর এই কমপাইল-টাইম টাইপ চেকিং কি আপনার কাছে পরিষ্কার হয়েছে, নাকি এটি কীভাবে JavaScript বা Python-এর চেয়ে আলাদা তা **তুলনামূলক উদাহরণ** দিয়ে দেখাব?